UNIVERSITY OF ENGINEERING & TECHNOLOGY

Peshawar, Pakistan

Lab Report

The while Loop, Nested while Loop & do...while Loop

STUDENT NAME	Shahzad Gul
REGISTRATION NO	0671
SUBJECT	Computer Programming (EE-170L)
LAB NUMBER	05
SEMESTER	Spring 2026

SPRING 2026 SESSION

05

Table of Contents

1. Objective

2. Theory

2.1 The while Loop

2.2 Nested while Loop

2.3 The do...while Loop

3. Lab Tasks and Solutions

Task 1: Numbers from 1 to 10 (while loop)

Task 2: Numbers from 1 to 10 (do...while loop)

Task 3: Sum using while Loop

4. Conclusion

1. Objective

The objectives of this laboratory session are to understand and apply the fundamental concepts of iterative control structures in C++ programming. Specifically, this lab focuses on:

- Understanding the **while loop** and its execution flow
- Implementing **nested while loops** for multi-level iteration
- Mastering the **do...while loop** and distinguishing it from the while loop
- Writing C++ programs that handle repetitive tasks using loop constructs

2. Theory

2.1 The while Loop

A `while` loop is a control flow statement that allows code to be executed repeatedly based on a given Boolean condition. The while loop consists of a block of code and a condition expression. The condition is first evaluated before any code within the block is executed. If the condition evaluates to `true`, the code within the loop body is executed. This process repeats until the condition becomes `false`.

The while loop is classified as an **entry-controlled loop** because the condition is checked at the beginning of each iteration. If the condition is initially `false`, the loop body may never execute.

Syntax of while Loop

```
while (condition)
{
    // statements to be executed
}
```

Component	Description
Condition	A Boolean expression evaluated before each iteration. If true, the loop body executes.
Loop Body	The block of statements executed repeatedly while the condition remains true.

Execution Flow:

1. The condition expression is evaluated.
2. If the condition is `true`, the loop body executes.
3. After the body executes, control returns to step 1.
4. If the condition is `false`, the loop terminates and control passes to the next statement after the loop.

2.2 Nested while Loop

A **nested while loop** is a while loop placed inside another while loop. For each iteration of the outer loop, the inner loop completes all of its iterations. This structure is useful for processing multi-dimensional data, generating patterns, and implementing algorithms requiring multiple levels of iteration control.

Syntax of Nested while Loop

```
while (outer_condition)          // Outer loop
{
    while (inner_condition)      // Inner loop
    {
        // inner loop statements
    }
    // outer loop statements
}
```

When using nested loops, it is essential to use different control variables for the inner and outer loops to avoid unexpected behavior and infinite loops.

2.3 The do...while Loop

The `do...while` loop is a variant of the while loop with one critical difference: the body of the loop is executed **before** the condition is checked. This guarantees that the loop body executes at least once, regardless of whether the condition is initially true or false.

The do...while loop is classified as an **exit-controlled loop** because the condition is evaluated at the end of each iteration rather than at the beginning.

Syntax of do...while Loop

```
do
{
    // statements to be executed
} while (condition);
```

Execution Flow:

1. The loop body executes first.
2. The condition expression is evaluated.
3. If the condition is `true`, the loop body executes again.
4. If the condition is `false`, the loop terminates.

Feature	while Loop	do...while Loop
Condition Check	Before iteration (entry-controlled)	After iteration (exit-controlled)
Minimum Executions	Zero (may never execute)	One (always executes at least once)
Semicolon	No semicolon after while()	Semicolon required after while(condition);

3. Lab Tasks and Solutions

Task 1: Display Numbers from 1 to 10 using while Loop

Problem Statement

Write a C++ program that uses a while loop to display numbers from 1 to 10, with each number on a separate line.

Solution

```
#include <iostream>
using namespace std;

int main()
{
    int i = 1;
    cout << "Numbers from 1 to 10:" << endl;
    while (i <= 10)
    {
        cout << i << endl;
        i++;
    }
    return 0;
}
```

EXPECTED OUTPUT

```
Numbers from 1 to 10:
1
2
3
4
5
6
7
8
9
10
```

Explanation: The variable `i` is initialized to 1 before the loop. The while loop checks if `i <= 10` before each iteration. Inside the loop, the current value of `i` is printed, and then `i` is incremented by 1. When `i` becomes 11, the condition becomes `false` and the loop terminates.

Task 2: Display Numbers from 1 to 10 using do...while Loop

> **Problem Statement**
>
> Repeat the first task using a do...while loop instead of a while loop.

Solution

```
#include <iostream>
using namespace std;

int main()
{
    int i = 1;
    cout << "Numbers from 1 to 10 (using do...while):" << endl;
    do
    {
        cout << i << endl;
        i++;
    } while (i <= 10);
    return 0;
}
```

```
EXPECTED OUTPUT
Numbers from 1 to 10 (using do...while):
1
2
3
4
5
6
7
8
9
10
```

Explanation: In the do...while version, the loop body executes first, printing the value of `i` and incrementing it. The condition `i <= 10` is checked after each iteration. Even if `i` were initialized to a value greater than 10, the body would still execute once before the condition is evaluated, which is the defining characteristic of the do...while loop.

Task 3: Sum Using while Loop

> **Problem Statement**
>
> Write a C++ program that prompts the user to enter a positive integer, then uses a while loop to calculate the sum of all numbers from 1 to the entered integer. Display the calculated sum.

Solution

```
#include <iostream>
using namespace std;

int main()
{
    int n;
    int i = 1;
    int sum = 0;

    cout << "Enter a positive integer: ";
    cin >> n;

    while (i <= n)
    {
        sum = sum + i;
        i++;
    }

    cout << "The sum of numbers from 1 to " << n << " is: " << sum << endl;
    return 0;
}
```

SAMPLE OUTPUT

```
Enter a positive integer: 5
The sum of numbers from 1 to 5 is: 15
```

Explanation: The program initializes `i` to 1 and `sum` to 0. The while loop continues as long as `i <= n`. In each iteration, the current value of `i` is added to `sum`, and then `i` is incremented. When the loop finishes, `sum` contains the total of all integers from 1 to `n`. For input 5, the calculation is 1 + 2 + 3 + 4 + 5 = 15.

4. Conclusion

In this laboratory session, the concepts of the `while` loop, nested `while` loop, and `do...while` loop were thoroughly explored through theoretical study and practical implementation. The key findings and learning outcomes are summarized below:

- The **while loop** is an entry-controlled loop where the condition is checked before each iteration. It is ideal for situations where the number of iterations is not known in advance.
- The **do...while loop** is an exit-controlled loop that guarantees the body executes at least once, making it suitable for menu-driven programs and input validation scenarios.
- **Nested while loops** allow for multi-level iteration patterns and are useful for generating patterns and processing multi-dimensional data.
- All three lab tasks were successfully implemented, demonstrating proficiency in applying iterative control structures to solve computational problems.

The understanding gained in this lab session provides a solid foundation for more advanced programming concepts, including array traversal, file processing, and algorithm implementation in subsequent coursework.